

DOCUMENTAÇÃO TÉCNICA

Diogo Avila Fraga
20103419

Trabalho de CLP – Integração entre Linguagens (Python + C)

1. Introdução

Este trabalho tem como objetivo demonstrar a integração entre duas linguagens de programação com vocações distintas, por meio do desenvolvimento de uma aplicação gráfica. A aplicação escolhida foi a geração do Fractal de Mandelbrot, um conjunto matemático amplamente utilizado para demonstrar conceitos de números complexos e iteração recursiva.

Como mencionado na descrição, o foco do trabalho não é a complexidade da aplicação gráfica em si, mas sim a implementação conjunta de duas linguagens diferentes, explorando suas características específicas e a comunicação entre elas.

2. Descrição da Aplicação

A aplicação desenvolvida gera e exibe o Fractal de Mandelbrot com base em parâmetros de resolução e número máximo de iterações.

O fractal é construído a partir da seguinte relação iterativa:

$$Z_{n+1} = Z_n^2 + C$$

Onde:

z e c são números complexos;

O ponto pertence ao conjunto se a sequência não divergir (módulo não ultrapassa 2).

Cada pixel da imagem representa um número complexo, e a quantidade de iterações necessárias para que o valor diverja determina a cor exibida.

3. Arquitetura da Aplicação

A aplicação foi dividida em dois módulos principais:

3.1 Módulo de Interface – Python

Responsabilidades:

Configuração dos parâmetros de geração.

- Criação da interface gráfica.

- Renderização da imagem.
- Chamada do serviço de cálculo.
- Gerenciamento da memória do array de resultados.
- Bibliotecas utilizadas:
- ctypes (integração com C)
- numpy (estrutura de dados numérica)
- matplotlib (renderização gráfica)

Python foi escolhido por sua simplicidade, legibilidade e facilidade de manipulação gráfica.

3.2 Módulo de Cálculo – C

Responsabilidades:

- Implementação do algoritmo matemático do Fractal de Mandelbrot.
- Processamento intensivo com loops aninhados.
- Manipulação direta de memória.
- Retorno dos valores de iteração por pixel.
- A linguagem C foi escolhida por:
 - Alto desempenho computacional.
 - Eficiência em operações matemáticas.
 - Controle de memória em baixo nível.

4. Método de Integração entre as Linguagens

A integração foi realizada utilizando uma biblioteca dinâmica (DLL).

4.1 Geração da DLL

O código em C foi compilado como biblioteca compartilhada:

```
gcc -shared -o mandelbrot.dll -fPIC mandelbrot.c
```

A função principal foi exportada utilizando:

```
#define EXPORT __declspec(dllexport)
```

4.2 Comunicação via ctypes

O Python utiliza o módulo ctypes para carregar a DLL:

```
lib = ctypes.CDLL("./mandelbrot.dll")
```

Os tipos dos argumentos são definidos explicitamente para garantir compatibilidade:

```
lib.calcular_mandelbrot.argtypes = [
    ctypes.c_int, ctypes.c_int,
    ctypes.c_double, ctypes.c_double,
    ctypes.c_double, ctypes.c_double,
    ctypes.c_int,
    ctypes.POINTER(ctypes.c_int)]
```

]

4.3 Compartilhamento de Memória

O Python cria um array NumPy:

```
resultado = np.zeros(largura * altura, dtype=np.int32)
```

Esse array é passado como ponteiro para o código C:

```
resultado.ctypes.data_as(ctypes.POINTER(ctypes.c_int))
```

O código C preenche diretamente esse espaço de memória, evitando cópias desnecessárias.

Esse método permite:

- Alta eficiência
- Baixa sobrecarga
- Comunicação direta entre as linguagens

5. Funcionamento do Algoritmo

Para cada pixel da imagem:

1. Converte-se a posição do pixel em um número complexo.
2. Inicializa-se $z = 0$
3. Aplica-se a fórmula iterativa.
4. Interrompe-se quando:
 - O módulo ultrapassa 2, ou
 - O número máximo de iterações é atingido.
5. Armazena-se o número de iterações no vetor resultado.

O valor armazenado determina a cor do pixel na renderização.

6. Estrutura do Projeto

Trabalho_CLP/

```
|
|— mandelbrot.c
|— interface.py
|— makefile
|— README.md
|— Documentacao.pdf
```

7. Justificativa da Escolha das Linguagens

A escolha das linguagens foi baseada em suas vocações distintas:

Linguagem	Função no Projeto	Característica Exploradas
Python	Interface e visualização	Produtividade e simplicidade
C	Processamento matemático	Desempenho e controle de memória

Essa divisão demonstra claramente a integração entre:

- Linguagem de alto nível (Python)

- Linguagem de baixo nível (C)

9. Conclusão

O trabalho demonstra com sucesso a integração entre duas linguagens de programação com finalidades distintas. A divisão entre interface e processamento matemático permitiu explorar:

- Modularização do sistema.
- Comunicação entre linguagens.
- Compartilhamento de memória.
- Uso de bibliotecas dinâmicas.

A solução evidencia como diferentes tecnologias podem ser combinadas para construir aplicações eficientes e organizadas.